MAKÜ
BURDUR MEHMET AKİF ERSOY ÜNİVERSİTESİ

VERİ YAPILARI VE ALGORİTMALAR

HAFTA 13
ARAMA ALGORİTMALARI VE AĞAÇLARI

Dr. Öğr. Üyesi Ebru AYDOĞAN

Bucak Bilgisayar ve Bilişim Fakültesi

Ders İçeriği: Hafta 13

- Doğrusal Arama
- İkili Arama
- Arama Ağaçları
  - İkili Arama Ağacı
  - AVL Ağacı

Doğrusal Arama

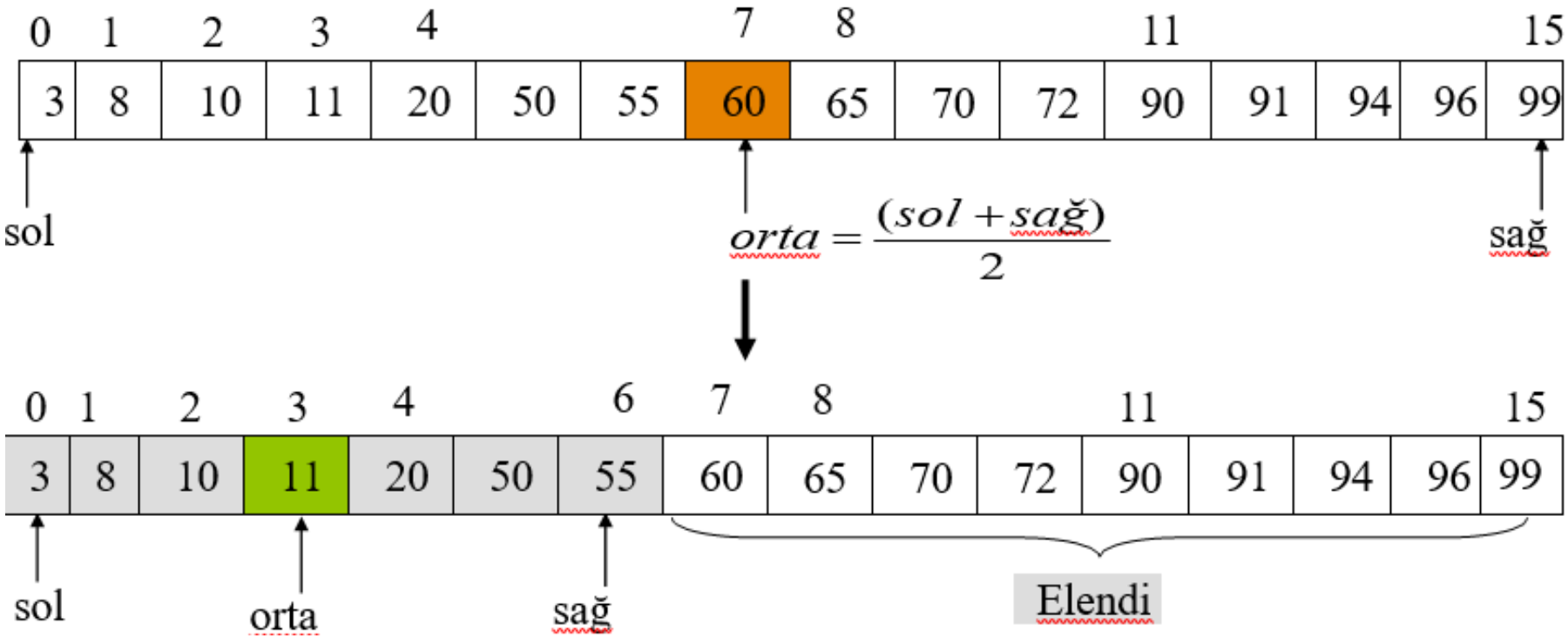
- Bir dizi (array) içinde bir verinin olup olmadığını anlamak için kullanılan basit bir algoritmadır.
- Doğrusal Arama Algoritması (Linear Search) aranan veriyi, dizinin ilk öğesinden başlayarak son öğesine doğru, her terimle tek tek karşılaştırır.
- Bu algoritma, dizi içinde aranana eşit olan bir terim bulursa onun indisini verir.
- Aranan veriye eşit olan terim bulamazsa -1 değerini verir.
- –1 değeri verdiğinde, aranan öğe dizi içinde değildir.
- Doğrusal arama algoritmasının zaman karmaşıklığı O(n)'dir.

İkili Arama

- İkili Arama (Binary Search), **sıralı** bir dizide, belirli değerin bulunmasına yönelik bir algoritmadır.
- Bu teknikteki her bir adımda, aranan değerin, dizinin orta değerine eşit olup olmadığı kontrol edilir.
- Eşit olmaması durumunda aranan değerin orta değer tarafından ikiye ayrılan kısımlardan hangisinde olduğu kontrol edilir.
- Aranan değeri içeren kısım bir sonraki adımda arama yapılacak dizi olur ve bu sayede arama yapılan listedeki eleman sayısı her adımda yarıya indirilmiş olur.
- Yani ortadaki eleman aranan elemana eşit değilse, aranan elemanın bulunduğu diğer yarı alanda arama işlemi tekrar edilir.
- Bu sayede her adımda arama uzayı yarıya indirilmiş olur.
- İkili Arama algoritmasının zaman karmaşıklığı O(log(n))'dir.

İkili Arama

- Örneğin aşağıda verilen dizi içerisinde **11** sayısının aranmasını inceleyelim.



Arama Ağaçları

- Bir veri yapısı içerisine çok sayıda (anahtar, değer) çiftleri saklamak istediğimizi varsayalım.
- Aşağıdaki işlemleri etkili bir şekilde yerine getirebilecek bir veri yapısına ihtiyacımız var.
  - Ekle(anahtar, değer)
  - Sil(anahtar, değer)
  - Bul(anahtar)
  - Min()
  - Max()
- Alternatif veri yapıları?
  - Dizi kullanmak
  - Bağlantılı liste kullanmak

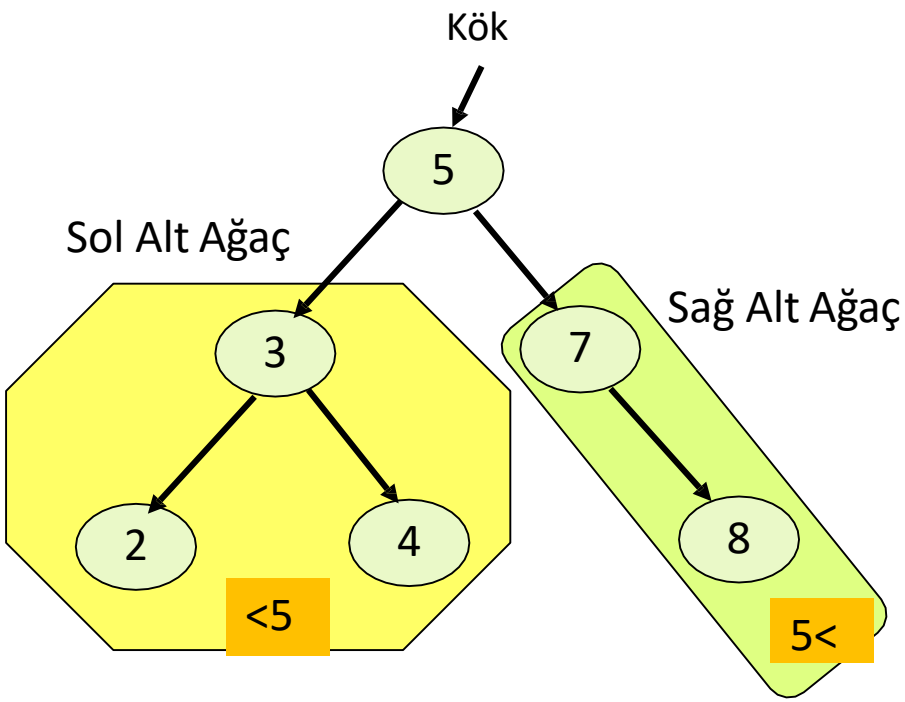
Arama Ağaçları

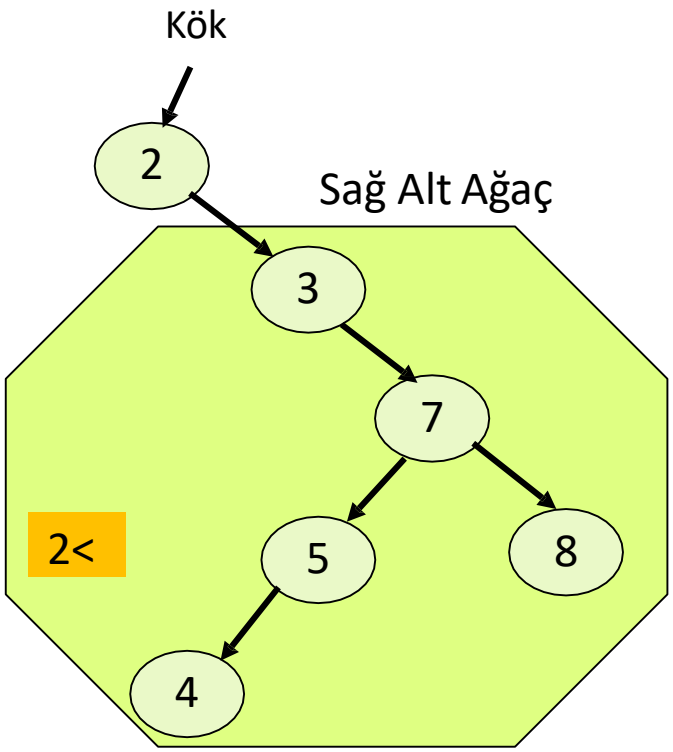
Kullanılan Verimli Arama Ağaçları

- Fikir: Verileri arama ağacı yapısına göre düzenlersek arama işlemi daha verimli olacaktır.
  - İkili Arama Ağacı (Binary Search Tree (BST))
  - AVL Ağacı
  - Splay Ağacı
  - 2-3-4 Ağacı
  - Red-Black Ağacı
  - B Ağacı ve B+ Ağacı

İkili Arama Ağacı

- İkili Arama Ağacı, her bir düğümdeki değerlere göre düzenlenir:
  - Sol alt ağaçtaki tüm değerler kök düğümünden küçüktür.
  - Sağ alt ağaçtaki tüm değerler kök düğümünden büyüktür.

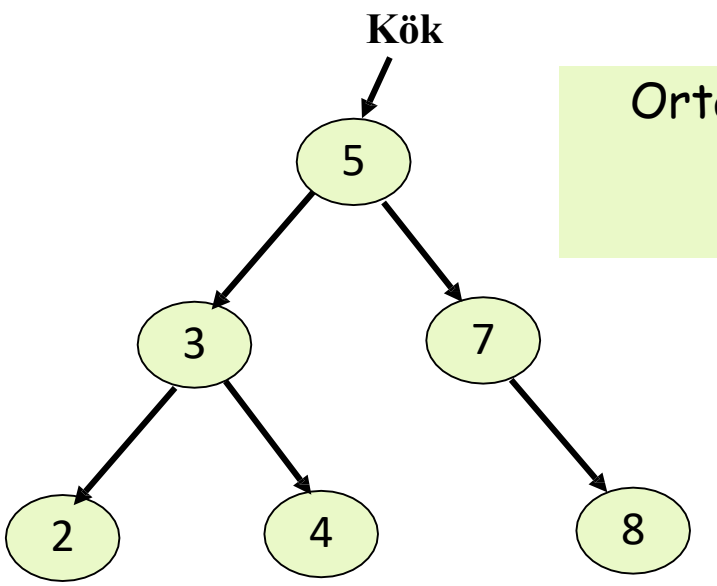




İkili Arama Ağacı

➢ İkili Arama Ağacında Sıralama

- İkili arama ağacının önemli özelliklerinden birisi Ortada-kök (Inorder) dolaşma algoritması ile düğümlere sıralı bir şekilde ulaşılmasını sağlar.

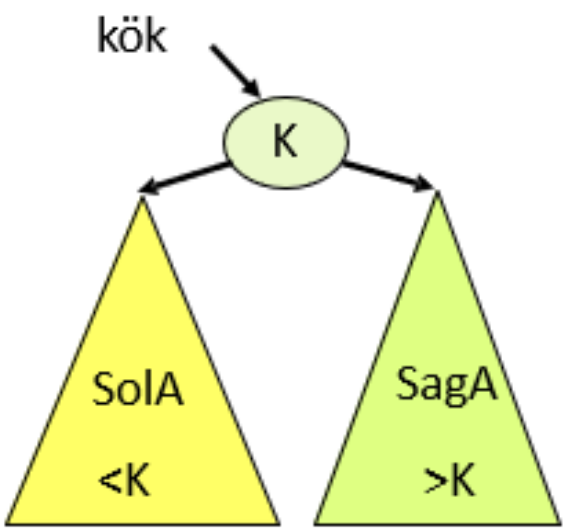


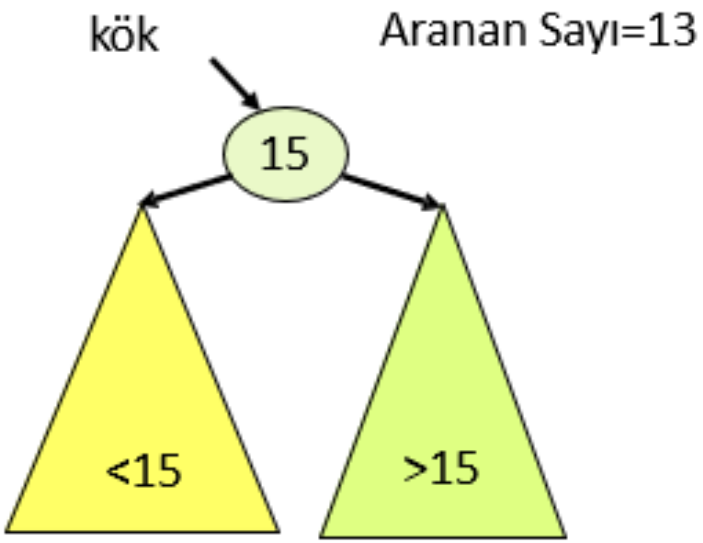
Ortada-kök sonucu
2 3 4 5 7 8

İkili Arama Ağacı

➢ İkili Arama Ağacı İşlemleri – Arama

- Değeri içeren düğümü bulma ve bu düğümü geri döndürme yapar.

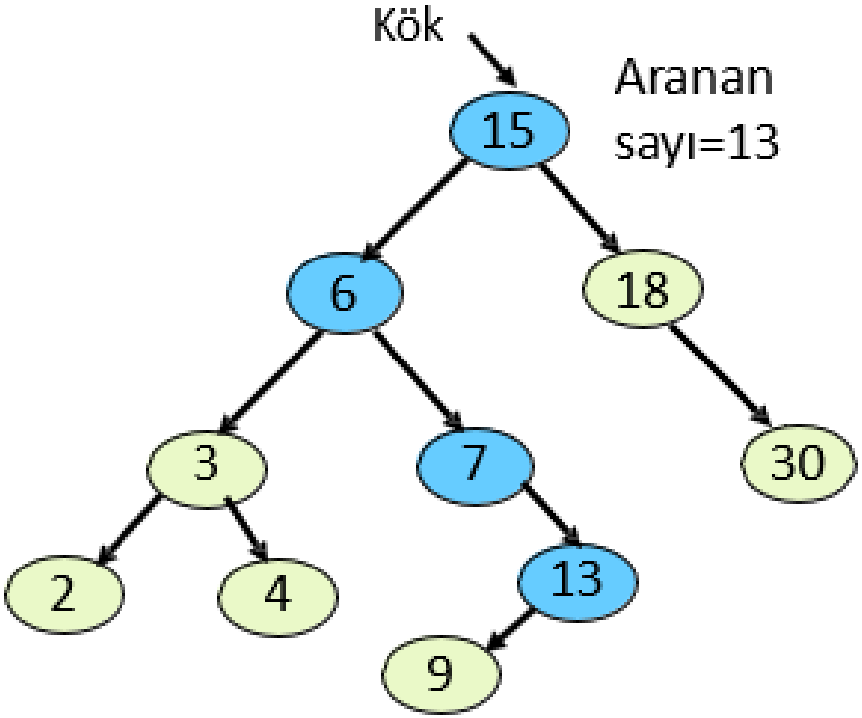





```
1.  Arama işlemine kökten başla
2.  if (aranaDeger == kok.deger)      return kok;
3.  if (aranaDeger < kok.deger)       Ara SolAltAğaç
4.  else                              Ara SagAltAğaç
```

İkili Arama Ağacı

➢ İkili Arama Ağacı İşlemleri – Arama

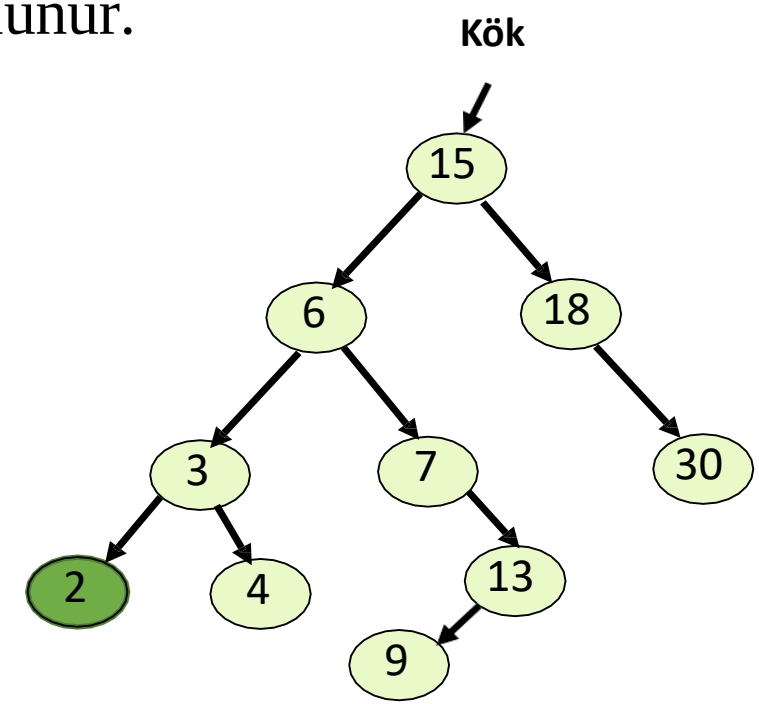


- Mavi renkli düğümler arama sırasında ziyaret edilen düğümlerdir.
- Algoritmanın çalışma karmaşıklığı O(d)'dir. (d = ağacın derinliği)

İkili Arama Ağacı

İkili Arama Ağacı İşlemleri – Min

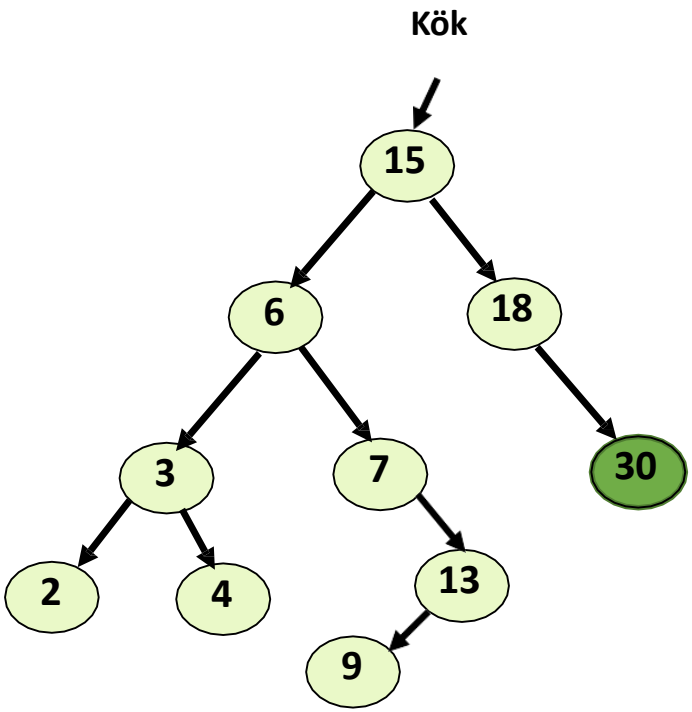
- Ağaçtaki en küçük elemanı içeren düğümü bulur.
  - En küçük elemanı içeren düğüm en soldaki düğümde bulunur.
  - Kökten başlayarak devamlı sola null oluncaya kadar gidilerek bulunur.



İkili Arama Ağacı

İkili Arama Ağacı İşlemleri – Max

- Ağaçtaki en büyük elemanı içeren düğümü bulur.
  - En büyük elemanı içeren düğüm en sağdaki düğümde bulunur.
  - Kökten başlayarak devamlı sağa gidilerek bulunur.



AVL Ağacı

1. AVL Ağacı Nedir?

- AVL ağacı, dengeli ikili arama ağacıdır.
- Adını geliştiricileri olan **Adelson-Velsky ve Landis**'ten alır.
- Her düğüm için sol alt ağaç ile sağ alt ağacın yüksekliği arasındaki fark (denge faktörü) en fazla 1 olabilir.

2. AVL Ağacı Özellikleri

- **Denge Faktörü**: Bir düğümdeki sol alt ağacın yüksekliği ile sağ alt ağacın yüksekliği arasındaki farktır.
  - Denge Faktörü = Sol Alt Ağaç Yüksekliği − Sağ Alt Ağaç Yüksekliği
  - Denge faktörü şu değerleri alabilir: **{-1, 0, 1}**
- AVL ağacı her ekleme veya silme işleminden sonra bu dengeyi korumak için yeniden düzenlenir.

AVL Ağacı

3. AVL Ağacında Dengeleme İşlemleri

AVL ağaçlarında dengeyi sağlamak için **rotasyon işlemleri** uygulanır.

a. Tek Rotasyon

- **Sağa Rotasyon (Right Rotation - RR)**: Sol dengesizliği düzeltmek için kullanılır.
- **Sola Rotasyon (Left Rotation - LL)**: Sağ dengesizliği düzeltmek için kullanılır.

b. Çift Rotasyon

- **Sol-Sağ Rotasyon (Left-Right - LR)**: Sol dengesizlik ve sağ alt ağaç büyümesi durumunda kullanılır.
- **Sağ-Sol Rotasyon (Right-Left - RL)**: Sağ dengesizlik ve sol alt ağaç büyümesi durumunda kullanılır.

AVL Ağacı

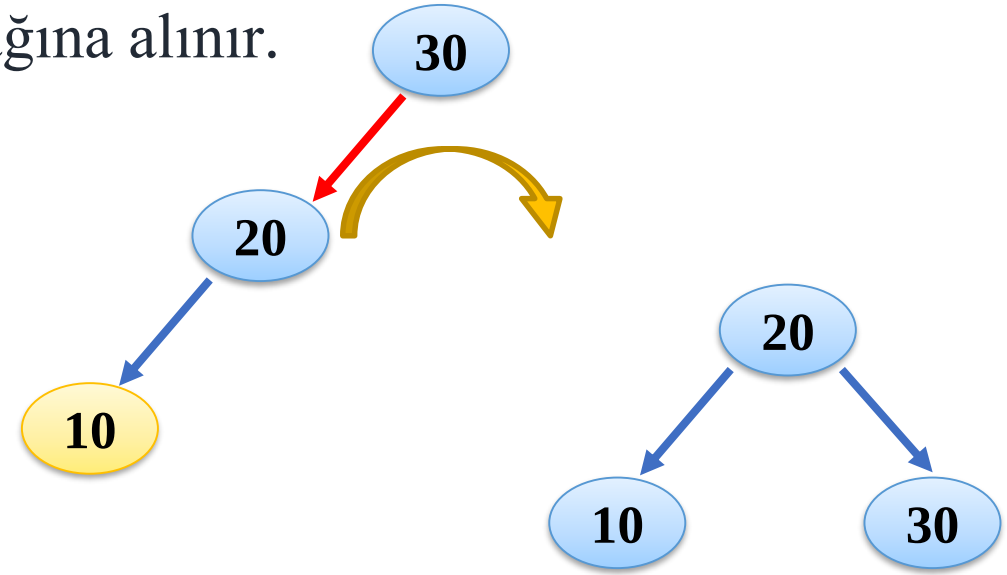
4. AVL Ağacında Temel İşlemler

- **Ekleme İşlemi**:
  - Yeni düğüm eklenir.
  - Denge faktörleri kontrol edilir.
  - Gerekirse rotasyon işlemleri uygulanır.
- **Silme İşlemi**:
  - Düğüm kaldırılır.
  - Denge kontrol edilir ve yeniden düzenleme yapılır.

➢ AVL ağaçları için O(log n) dir.

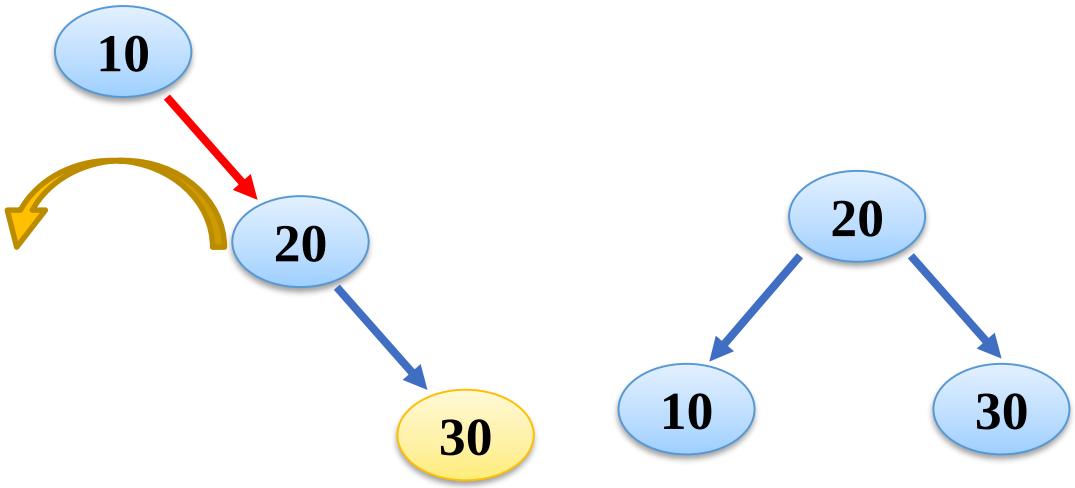
AVL Ağacı

- **Sağa Rotasyon (Right Rotation - RR):**
  - Sol alt ağaç daha yüksek, denge sağlamak için sağa rotasyon gerekir.
  - 30 kökü, 20'nin sağına alınır.
  - 20, yeni kök olur.



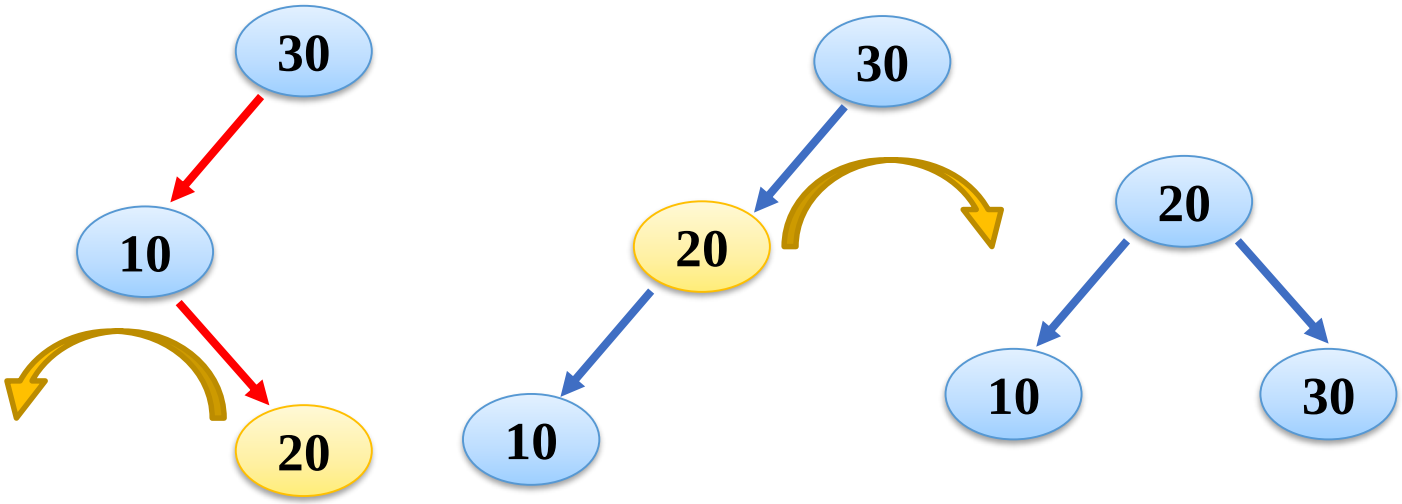
AVL Ağacı

- **Sola Rotasyon (Left Rotation - LL):**
  - Sağ alt ağaç daha yüksek, denge sağlamak için sola rotasyon gerekir.
  - 10 kökü, 20'nin soluna alınır.
  - 20, yeni kök olur.



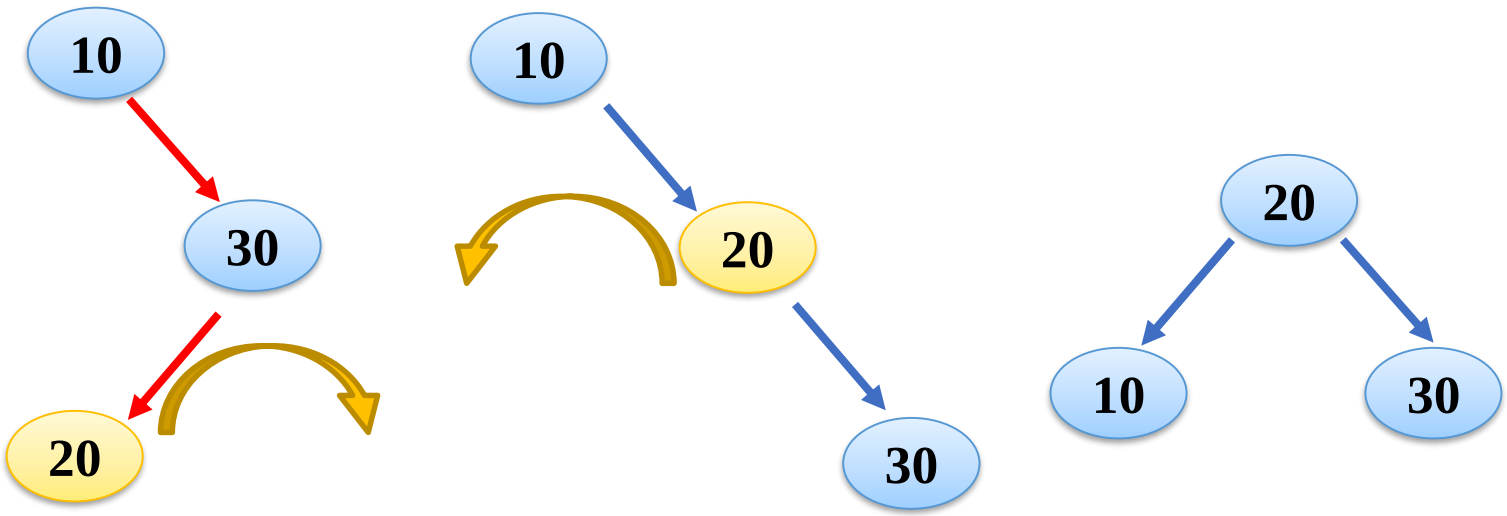
AVL Ağacı

- **Sol-Sağ Rotasyon (Left-Right Rotation - LR):**
  - Sol alt ağaçta sağa kayma var.
  - İlk olarak **sol alt ağaçta sağa kayma** olduğu için 10 ve 20 arasında sola rotasyon yapılır.
  - Daha sonra **kökte sağa rotasyon** yapılır.



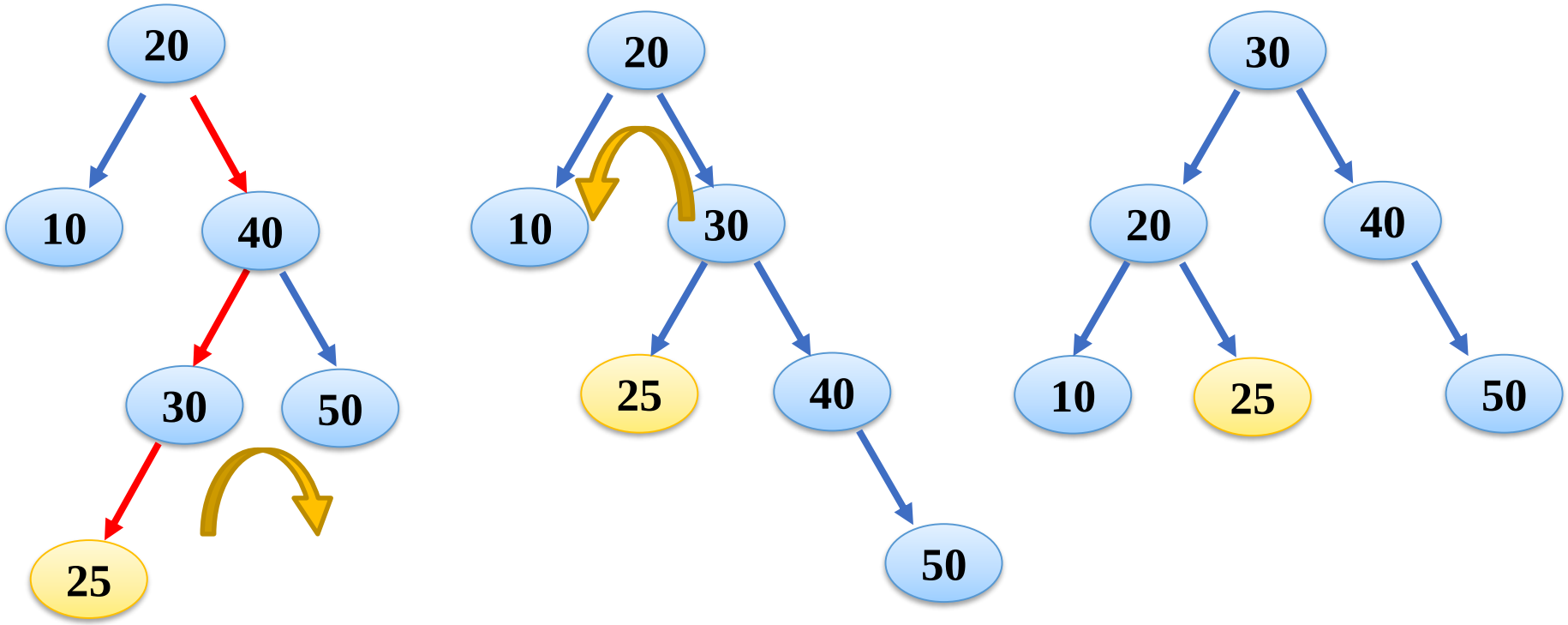
AVL Ağacı

- **Sağ-Sol Rotasyon (Right-Left Rotation - RL):**
  - Sağ alt ağaçta sola kayma var.
  - İlk olarak **sağ alt ağaçta sola kayma** olduğu için 30 ve 20 arasında sola rotasyon yapılır.
  - Daha sonra **kökte sola rotasyon** yapılır.



AVL Ağacı

- **Örnek:** 10, 20, 30, 40, 50, 25 sayılarını sırasıyla AVL ağacına ekleyelim.

AVL Ağacı

- **Örnek:** 10, 20, 30, 40, 50, 25 sayılarını sırasıyla AVL ağacına ekleyelim.